מספר זהות:

סמסטר א' תשפ"ו
מועד: א' 02/02/2026 9:00
משך המבחן: שעתיים ועשרים דקות
חומר עזר: כל חומר עזר כתוב מותר

בחינה בקורס: **תכנות מכוון עצמים ושפת ++C**
מרצים: **אמיר קירש, ד"ר איריס גבר**

קטעי הקוד זהים לכל הטורים במבחן
בתום המבחן חובה להחזיר את קטעי הקוד ביחד עם שאלון המבחן

הנחיות כלליות לבחינה:

- המבחן מורכב מחלק אחד ובו:
  - **10 שאלות אמריקאיות. משקל כל שאלה 10 נקודות, סה"כ: 100 נק'.**
  - **4 שאלות אמריקאיות בונוס. משקל כל שאלה 4 נקודות, סה"כ: 16 נק'.**
  - **לא ניתן לצבור יותר מ-100 נקודות במבחן.**
- יש לסמן תשובה אחת לכל שאלה בטופס התשובות. במידה ומספר תשובות נראות נכונות יש לסמן את התשובה הנכונה ביותר.
- בסיום המבחן יש לרשום מס' ת.ז. על גבי טופס המבחן ולוודא שטופס התשובות מוגש יחד עם טופס המבחן.
- המבחן הינו עם חומר פתוח. כל חומר עזר כתוב מותר למעט מכשירים אלקטרוניים למיניהם. אין להעביר חומר עזר בין תלמידים במהלך המבחן.

בהצלחה !

קטע קוד A

```
 1. struct Shape {
 2.   Shape() { /* print(); */ }
 3.   virtual double area() const = 0;
 4.   virtual Shape* clone() const = 0;
 5.   virtual void print() const { cout << "Shape\n"; }
 6.   virtual ~Shape() { /* cout << area() << endl; */ }
 7. };
 8.
 9. class Rect : public Shape {
10.   int w, h;
11. public:
12.   Rect(int w, int h) : w(w), h(h) {}
13.   double area() const override { return w * h; }
14.   Shape* clone() const override { return new Rect(*this); }
15.   void print() const override {
16.     cout << "Rect(" << w << ", " << h << ")\n";
17.   }
18. };
19.
20. class Circle : public Shape {
21.   int r;
22. public:
23.   Circle(int r) : r(r) {}
24.   double area() const override { return 3 * r * r; }
25.                         // Pi = 3 here ^ is good enough for us...
26.   Shape* clone() const override { return new Circle(*this); }
27.   void print() const override { cout << "Circle(" << r << ")\n"; }
28. };
29.
30. class ShapeBox {
31.   Shape* p;
32.   void copy(const Shape* other) {
33.     p = (other ? other->clone() : nullptr);
34.   }
35. public:
36.   ShapeBox(const Shape* s = nullptr) { copy(s); }
37.   ~ShapeBox() { delete p; }
38.   ShapeBox(const ShapeBox& other) { copy(other.p); }
39.   ShapeBox& operator=(const ShapeBox& other) {
40.     if (this != &other) {
41.       delete p;
42.       copy(other.p);
43.     }
44.     return *this;
45.   }
46.   double area() const {
47.     return p ? p->area() : 0;
48.   }
```

```
  void print() const {
    if (p) p->print();
    else cout << "empty\n";
  }
};

template<class Container, class F>
void applyOn(const Container& c, F f)  {
  for (const auto& item : c)
    f(item);
}

template<class ShapeBoxContainer>
double sumArea(const ShapeBoxContainer& c) {
  double s = 0;
  applyOn(c, [&s](const auto& item) {
    s += item.area();
  });
  return s;
}
```

להלן מספר main אפשריים:

A1 main

```
int main() {
  list<Shape*> shapes1;
  shapes1.push_back(new Circle(1));
  shapes1.push_back(new Rect(3, 5));

  applyOn(shapes1, [](auto pshape) {
    pshape->print();
  });

  list<ShapeBox> shapes2;
  applyOn(shapes1, [&shapes2](auto pshape) {
    shapes2.push_back(pshape);
  });

  cout << sumArea(shapes2) << endl;

  applyOn(shapes1, [](auto pshape) {
    delete pshape;
  });
}
```

המשך בעמוד הבא!

A2 main

```
int main() {
  vector<ShapeBox> shapes = {
    new Circle(3),
    new Rect(1, 2)
  };
  applyOn(shapes, [](const auto& box) {
    box.print();
  });
}
```

A3 main

```
int main() {
  Shape* ps = new Rect(1, 1);
  ShapeBox c_box = dynamic_cast<Circle*>(ps);
  cout << c_box.area() << endl;
  delete ps;
}
```

קטע קוד B

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.  static Groups groups;
 6.  static Groups::iterator joinGroup(const string& s) {
 7.    auto itr = groups.find(s);
 8.    if(itr == groups.end()) {
 9.      // see explanation regarding map::insert at line 38
10.       itr = groups.insert({s, 0}).first;
11.     }
12.     ++(itr->second);
13.     return itr;
14.   }
15.   static void leaveGroup(Groups::iterator itr) {
16.     if(--(itr->second) == 0) groups.erase(itr);
17.   }
18.   // this is how we actually hold our string:
19.   Groups::iterator s_itr;
20. public:
21.   MyString(const string& str): s_itr(joinGroup(str)) {}
22.   MyString(const char* str = ""): s_itr(joinGroup(str)) {}
23.   ~MyString() { leaveGroup(s_itr); }
24.   MyString(const MyString& s): s_itr(s.s_itr) { s_itr->second++; }
25.   MyString& operator=(const MyString& s) {
26.     leaveGroup(s_itr);
27.     s_itr = s.s_itr;
28.     s_itr->second++;
29.     return *this;
30.   }
31.   operator const char*() const { return s_itr->first.c_str(); }
32.   size_t count() const { return s_itr->second; }
33. };
34.
35. // in MyString.cpp
36. Groups MyString::groups;
37.
38. // explanation regarding map::insert (line 10):
39. // map.insert returns std::pair<iterator, bool>
40. // 'first' is an iterator pointing to the new item or to
41. // an item with the same key that already existed, which
42. // prevented the insertion
43. // 'second' is whether insertion was performed
44.
45. // code continues on next page
46.
47.
```

```
48. int main() {
49.   std::array<MyString, 3> sarr;
50.   sarr[0] = "hi";
51.   sarr[1] = sarr[0];
52.   auto is_single = [](const auto& s) { return s.count() == 1; };
53.   cout << count_if(sarr.begin(), sarr.end(), is_single) << endl;
54.   sarr[2] = sarr[1] = "bye";
55.   cout << count_if(sarr.begin(), sarr.end(), is_single) << endl;
56.   cout << sarr[1] << endl;
57. }
```

קטע קוד C – לשאלות הבונוס

```
 1. struct Shape {
 2.   virtual Shape* clone() const = 0;
 3.   virtual ~Shape() {}
 4.   const char* type = nullptr;
 5.   friend ostream& operator<<(ostream& out, const Shape& s) {
 6.     return out << s.type;
 7.   }
 8. };
 9.
10. struct Rect : public Shape {
11.   Rect() { type = "Rect"; }
12.   Shape* clone() const override { return new Rect(*this); }
13. };
14.
15. struct Circle : public Shape {
16.   Circle() { type = "Circle"; }
17.   Shape* clone() const override { return new Circle(*this); }
18. };
19.
20. class ShapeBox {
21.   Shape* p = nullptr;
22.   void copy(const Shape* other) {
23.     cout << "calling clone\n";
24.     p = other->clone();
25.   }
26. public:
27.   ShapeBox(Shape* s = nullptr) { if(s) copy(s); }
28.   ~ShapeBox() {
29.       cout << "in dtor for: " << *this << endl;
30.       delete p;
31.   }
32.   ShapeBox(ShapeBox&& other) noexcept {
33.     cout << "move ctor\n";
34.     std::swap(p, other.p);
35.   }
36.   ShapeBox& operator=(ShapeBox&& other) noexcept {
37.     cout << "move assignment\n";
38.     std::swap(p, other.p);
39.     return *this;
40.   }
41.   friend ostream& operator<<(ostream& out, const ShapeBox& s) {
42.     if (s.p) out << *(s.p);
43.     else out << "empty";
44.     return out;
45.   }
46. };
47.
```

```
48. int main() {
49.   Rect r;
50.   Circle c;
51.   ShapeBox box(std::move(&c));
52.   std::array<ShapeBox, 3> shapes;
53.   shapes[0] = std::move(box);
54.   shapes[2] = &r;
55.   for(auto& s : shapes) {
56.       cout << s << endl;
57.   }
58. }
```

Following are possible outputs of the above program
(to be used in the exam questions 11-14):

```
(a)  Rect
(b)  Circle
(c)  empty
(d)  calling clone
(e)  move ctor
(f)  move assignment
(g)  in dtor for: Rect
(h)  in dtor for: Circle
(i)  in dtor for: empty
```

בהצלחה !